

A Multi-Agent Generative AI Framework for Customer Support Ticket Resolution using Langchain & LangGraph

A PROJECT REPORT

Submitted by

BL.SC.P2DSC24028

Siva Jyothis

MASTER OF TECHNOLOGY

IN

DATA SCIENCE



AMRITA SCHOOL OF COMPUTING, BENGALURU

AMRITA VISHWA VIDYAPEETHAM

BENGALURU 560 035

MAY 2026

AMRITA VISHWA VIDYAPEETHAM

AMRITA SCHOOL OF COMPUTING, BENGALURU, 560035



BONAFIDE CERTIFICATE

This is to certify that the dissertation report entitled “A Multi-Agent Generative AI Framework for Customer Support Ticket Resolution using Langchain & LangGraph” submitted by

BL.SC.P2DSC24028

Siva Jyothis

in partial fulfillment of the requirements as part of **Master of Technology** in “**DATA SCIENCE**” is a bonafide record of the work carried out under my guidance and supervision at Amrita School of Computing, Bengaluru.

Dr. Suja P.

Professor

Guide

School of Computing

Dr. Gopalakrishnan E.A.

Chair

School of Computing

This project report was evaluated by us on

Examiner 1

Examiner 2

ACKNOWLEDGEMENTS

The satisfaction that accompanies successful completion of any task would be incomplete without mention of people who made it possible, and whose constant encouragement and guidance have been source of inspiration throughout the course of this project work.

I offer my sincere pranams at the lotus feet of “**AMMA**”, **MATA AMRITANANDAMAYI DEVI** who showered her blessing upon me throughout the course of this project work.

I owe my gratitude to **Prof. Manoj P.**, Director, Amrita Vishwa Vidyapeetham Bengaluru Campus. I thank **Dr. Gopalakrishnan E.A.**, Principal and Head, Amrita School of Computing and Amrita School of Artificial Intelligence, Bengaluru for his support and inspiration.

It is a great pleasure to express my gratitude and indebtedness to my project guide **Suja P.**, Professor, Department of Computer Science and Engineering, Amrita School of Computing, Bengaluru for her valuable guidance, encouragement, moral support, and affection throughout the project work.

I would like to express my gratitude to project panel members for their suggestions, encouragement, and moral support during the process of project work and all faculty members for their academic support. Finally, I am forever grateful to my parents, who have loved, supported and encouraged me in all our endeavors.

ABSTRACT

Customer support ticket management plays an important role in modern IT Service Management (ITSM) systems. Manual ticket automation processes can be cumbersome to manage and handle contextual reasoning, semantic routing, or automated response generation. To address these limitations, This project presents a multi-agent Generative AI framework for intelligent customer support ticket automation using LangChain and LangGraph. This automation strategy encompasses the entire ticket handling workflow, which includes classifying both the ticket type as well as predicting the priority, routing tickets to the appropriate queue, and generating contextually aware responses. Multiple specialized LLM Agents are utilized sequentially within a shared state pipeline to provide a modular and explainable decision-making process. This system combines Retrieval-Augmented Generation (RAG) with FAISS vector databases and sentence embeddings for enhanced semantic comprehension and routing precision. Pre-trained open-source Large Language Models like Mistral, LLaMA 2, and Phi-3 are used for efficient local inference without fine-tuning. The backend is built with FastAPI, the frontend is created with Streamlit, SQLite is employed to deal with the ticket history, and Docker is utilized for containerized deployment. Experimental results with respect to Accuracy, Precision, Recall, F1-Score, BERTScore and Semantic Similarity showed that the proposed multi-agent architecture successfully enhances intelligent ticket processing and automated response generation in an end-to-end IT Service Management process.

TABLE OF CONTENTS

	Page No.
ACKNOWLEDGEMENTS	ii
ABSTRACT	iv
LIST OF FIGURES	viii
LIST OF TABLES	ix
CHAPTER 1- INTRODUCTION	
1.1 INTRODUCTION TO INTELLIGENT CUSTOMER SUPPORT AUTOMATION	1
1.2 MOTIVATION	3
1.3 PROBLEM STATEMENT	4
1.4 OBJECTIVES	5
CHAPTER 2 – LITERATURE SURVEY	
2.1 MACHINE LEARNING AND NLP-BASED TICKET CLASSIFICATION SYSTEMS	6
2.2 MULTI-AGENT AI AND LLM-BASED DECISION -MAKING SYSTEMS	7
2.3 PROMPT ENGINEERING, AGENT ARCHITECTURES AND RETRIEVAL-AUGMENTED SYSTEMS	8
2.4 RESEARCH ON PROMPT OPTIMIZATION AND LLM WORKFLOW EFFICIENCY	9
2.5 EMERGING MULTI-AGENT LLM FRAMEWORKS FOR INTELLIGENT AUTOMATI	10
CHAPTER 3 – SYSTEM REQUIREMENTS AND ANALYSIS	
3.1 SOFTWARE REQUIREMENTS	
3.1.1 DEVELOPMENT AND PROGRAMMING TOOLS	11

3.1.2	AI FRAMEWORKS AND MODEL INTEGRATION	12
3.1.3	BACKEND, FRONTEND, AND DATABASE TOOLS	13
CHAPTER 4 – SYSTEM DESIGN		
4.1	INTRODUCTION TO SYSTEM DESIGN	14
4.2	OVERALL SYSTEM WORKFLOW	15
4.3	PREPROCESSING AND INPUT PROCESSING LAYER	16
4.4	MULTI-AGENT PROCESSING ARCHITECTURE	17
4.5	SHARED STATE MANAGEMENT AND LANGGRAPH ORCHESTRATION	18
4.6	SEMANTIC RETRIEVAL AND RESPONSE GENERATION	19
4.7	BACKEND COMMUNICATION, DATABASE INTEGRATION, AND FRONTEND VISUALIZATION	19
CHAPTER 5 – SYSTEM IMPLEMENTATION		
5.1	MODULES USED WITH DESCRIPTION	
5.1.1	INPUT AND PREPROCESSING MODULE	20
5.1.2	ISSUE TYPE CLASSIFICATION MODULE	22
5.1.3	PRIORITY PREDICTION MODULE	23
5.1.4	QUEUE ROUTING MODULE	24
5.1.5	RESPONSE GENERATION MODULE	25
5.1.6	LANGGRAPH WORKFLOW ORCHESTRATION MODULE	26
5.1.7	FASTAPI BACKEND MODULE	27

5.1.8	SQLITE DATABASE MODULE	27
5.1.9	FRONTEND USER INTERFACE MODULE	28
5.1.10	DOCKER DEPLOYMENT MODULE	29
5.2	IMPLEMENTATION WORKFLOW	30
CHAPTER 6 – SYSTEM TESTING		
6.1	PERFORMANCE METRICS FOR CLASSIFICATION TASKS	32
6.1.1	ACCURACY	33
6.1.2	PRECISION	33
6.1.3	RECALL	33
6.1.4	F1-SCORE	34
6.2	RESPONSE GENERATION EVALUATION	34
6.2.1	ROUGE-L EVALUATION	35
6.2.2	BERTSCORE EVALUATION	35
CHAPTER 7 – RESULTS AND ANALYSIS		
7.1	TYPE AGENT RESULTS AND ANALYSIS	37
7.2	PRIORITY AGENT RESULTS AND ANALYSIS	39
7.3	QUEUE AGENT RESULTS AND ANALYSIS	41
7.4	RESPONSE GENERATION RESULTS AND ANALYSIS	43
7.5	COMPARATIVE DISCUSSION	45
7.6	USER INTERFACE AND DEPLOYMENT RESULTS	46
CHAPTER 8 – CONCLUSION AND FUTURE ENHANCEMENT		
8.1	CONCLUSION	47
8.2	FUTURE SCOPE	48
REFERENCES		

LIST OF FIGURES

Fig 4.1	The Hybrid RAG-Based Legal Question Answering Framework System Architecture	15
Fig 5.1	Implementation Workflow	30
Fig 7.1	Type Agent Accuracy Comparison	38
Fig 7.2	Priority Agent Accuracy Comparison	40
Fig 7.2	Priority Agent Accuracy Comparison	42
Fig 7.4	BERTScore Comparison & Semantic Similarity of Response Models	44
Fig 7.6	User Interface and Deployment Screens	46

LIST OF TABLES

Table 7.1	Type Agent Classification Results	40
Table 7.2	Priority Agent Classification Results	42
Table 7.3	Queue Agent Classification Results	44
Table 7.4	Response Generation Evaluation Results	46

CHAPTER - 1

INTRODUCTION

1.1 Introduction to Intelligent Customer Support Automation

Customer support and IT service management systems play a critical role in modern organizations. Many customer queries and technical support requests are received by enterprises from various domains including banking, healthcare, education, e-commerce, cloud computing, software services, etc. on a daily basis. The requests of this support are typically made via tickets with issue description, complaints, information requests, billing problems, software issues, or service problems. Efficient handling of these tickets is essential to maintain customer satisfaction, ensure operational continuity, and improve overall business productivity. The customer support system is mostly dependent on manual ticket processing or rule-based automation methods. In manual workflows, support staff have to read every ticket, assign it to the right category, rate its urgency, assign it to the appropriate department, and compose a reply. Manual handling becomes time-consuming, costly and prone to errors with increasing ticket volume. The longer tickets take to process, the more impact it has on the customer experience and the burden on customer service teams. To overcome these challenges, many organizations have adopted machine learning and Natural Language Processing (NLP)-based automation systems. Previous works adopted mainly single task classification of tickets using supervised machine learning algorithms like Support Vector Machines (SVM), Naive Bayes, Random Forests (RF), and deep learning models. While these methods improved automation to some extent, they were limited in understanding contextual semantics, reasoning capabilities, and handling complex ticket variations. Most traditional systems dealt with support tickets

as standalone classification tasks without acknowledging relationships between several decision making tasks like issue classification, priority estimation, routing and reply generation. Recent advances in Large Language Models (LLMs) and Generative AI have significantly transformed intelligent automation systems. The models like LLaMA, Mistral, GPT, and Phi-series models exhibit excellent language comprehension, semantic reasoning, and context-aware text generation skills. These models can understand free-form user queries and perform various decision-making tasks using prompt engineering techniques. Yet centralized LLM-based systems suffer from certain shortcomings, such as a lack of modularity, lack of explainability, high computational costs, and challenges in maintaining enterprise applications with deterministic workflows. To overcome these drawbacks, multi-agent AI systems have become a promising solution for complex task automation. In multi-agent architecture, each agent is a single agent that works on a specific task independently of others, but in a shared flow this enhances interpretability, modularity, scalability, and transparency of decisions. This project proposes a Multi-Agent Generative AI Framework for Customer Support Ticket Resolution using LangChain and LangGraph. The framework breaks down the ticket handling tasks into a number of intelligent agents that are chained into a deterministic workflow pipeline. The system integrates prompt-engineered Large Language Models, Retrieval-Augmented Generation (RAG), FAISS vector databases, and semantic retrieval techniques to create an end-to-end intelligent ticket automation framework.

The proposed system contains four major AI agents:

- Issue Type Agent
- Priority Prediction Agent
- Queue Routing Agent
- Response Generation Agent

Every agent carries out a distinct decision-making function and update shared state space via LangGraph. The framework is orchestrated using LangChain and integrates local LLM. Semantic retrieval with Sentence Transformers and FAISS vector similarity search during inference to retrieve semantically similar examples of tickets enhances contextual understanding and routing accuracy. This pipeline also features a FastAPI back-end for real-time API communication, Streamlit front-end for user interaction, SQLite database integration for maintaining ticket history, and Docker containerization for portability.

The architecture as a whole showcases the potential of modern Generative AI systems in the context of IT Service Management (ITSM) practices, highlighting their ability to streamline customer support operations effectively. The proposed framework aims to enhance the accuracy of ticket handling, to minimize manual tasks, to allow for scalable support automation and to offer explainable multi-stage AI decision making. Multi-agent orchestration, semantic retrieval, local LLM inference, and real-time deployment technologies enable the system to create a full-fledged intelligent customer support automation platform.

1.1 Motivation

Modern organizations receive a massive number of customer support and IT service tickets every day related to technical issues, billing problems, account management, service outages, and product inquiries. As ticket volume grows substantially, manual process become time-consuming, costly, and prone to inconsistent prioritization, routing, and response times. Traditional machine learning and rule-based automation systems struggle to grasp the semantic meaning, relations in context, and differences in the natural language of incoming tickets. While recent progress in Large Language Models (LLMs) and Generative AI has demonstrated their impressive abilities in

understanding and reasoning with natural language, many centralized LLM platforms are still not modular, interpretable, and have structured workflow control for enterprise applications. This encouraged the creation of a multi-agent AI system that breaks down ticket processing into intelligent agents for classification, priority estimation, routing and response generation. The combination of LangGraph, LangChain, the FAISS locally deployable semantic retrieval, and open-source LLMs brings together a scalable, explainable, and privacy-focused customer support automation system.

1.2 Problem Statement

Customer support tickets are highly unstructured, context-rich, and linguistically diverse, making automated ticket understanding and processing a challenging task for conventional systems. The current ticket automation methods primarily target single classification or machine learning pipelines that are tightly coupled and manage to correctly recognize only few attributes of ticket from an unstructured text. Most chatbot-oriented LLM systems focus on writing responses to conversational inputs but give no indication of structured reasoning or explainable decision-making processes, and rule-based systems are not easily adapted to real-world variations in language. Moreover, most existing multi-agent AI frameworks does not support the deterministic sequential state passing between specialized agents for enterprise IT Service Management applications. Most of the current systems are also not designed to provide a unified architecture for semantic retrieval, local LLM inference, REST-based deployment, database management, and interactive visualization on the frontend. Thus, It is essential to develop an intelligent multi-agent system that can execute structured ticket analysis, semantic reasoning, retrieval-augmented routing and automated responses in a coordinated and explainable workflow pipeline for real-world customer support.

1.3 Objectives

The main objectives of the proposed project include:

1. To design and develop a multi-agent AI framework for intelligent customer support ticket automation using LangChain and LangGraph.
2. To implement designated AI agents for:
 - Ticket type classification
 - Priority prediction
 - Queue routing
 - Context-aware response generation
3. To create deterministic sequential workflow pipeline with LangGraph and Langchain for structured state transfer between agents.
4. To incorporate Retrieval-Augmented Generation (RAG) with FAISS vector databases and sentence embeddings for semantic similarity retrieval.
5. Efficiently use open-source Large Language Models like Mistral, LLaMA 2, and Phi-3 for local inference without complete fine-tuning.
6. To create backend system using FastAPI to interact with RsetAPI and make predictions for the tickets in real-time.
7. To create an interactive front-end interface that uses Streamlit to submit tickets and visualize the AI-generated results.
8. Implement ticket history management using SQLite to record tickets which have been processed.
9. To evaluate system performance using metrics such as Accuracy, Precision, Recall, F1-Score, BERTScore, and Semantic Similarity.
10. To create a complete end-to-end intelligent IT Service Management automation framework capable of real-world deployment..

CHAPTER - 2

LITERATURE REVIEW

In today's world with the growing number of digital services and online customer interaction platforms, customer support automation has emerged as a crucial research field. Organizations get a huge amount of tickets for technical issues, billing problems, product issues, account services and operational questions. The manual handling of these tickets can result in delayed responses, misrouted tickets, and expensive operations. To overcome these limitations, researchers have investigated the use of Machine Learning (ML), Deep Learning (DL), Natural Language Processing (NLP), Large Language Models (LLMs), and multi-agent AI systems for intelligent ticket handling and workflow automation. This chapter summarizes the research works conducted in the fields of IT ticket classification, customer support automation, multi-agent AI systems, prompt engineering, and retrieval enhanced LLM workflows. The literature survey is divided into four major sections based on the research focus of previous studies.

2.1 Machine Learning and NLP-Based Ticket Classification Systems

The initial research into customer support automation was largely centered on supervised machine learning approaches to categorize tickets and forecast issues. These systems were designed to use structured text processing and statistical learning techniques to automate tasks of ticket classification. Leonardo S. B. Pereira et al [1]. proposed a machine learning-based framework for IT support ticket classification. The study proposed a prototype of a deployable system that can classify ticket topics by supervised learning methods. The work illustrated the potential of ML techniques in categorization of tickets in IT service environments. however, the system was

based on fixed labels and fixed ticket categories, and did not intend for urgency estimation, routing across multiple ticket attributes. Dalubuhle Dlodlo and Khulekani Sibanda [2]. investigated automated ticket classification for IT helpdesks using machine learning algorithms combined with hyperparameter optimization techniques. Their work evaluated multiple classifiers for improving helpdesk ticket prediction accuracy. The study enhanced the accuracy of classification but the system did not support multi-step reasoning, response generation, and semantic contextual understanding. Md. Shamim Towhid et al [3]. proposed a token-prioritization approach for addressing the significant class imbalance problem in network-change ticket classification. Proposed method resulted in enhancement of supervised classification performance by adding importance to salient textual tokens in the classification process. The method resolved data imbalance issues well, however, it was restricted to typical classification tasks and lacked semantic reasoning, contextual retrieval and capabilities of priority or queue inference. These studies show that traditional ML and NLP techniques are suitable for single-stage ticket classification problems, but do not support explainable multi-stage decision-making processes demanded by modern intelligent IT Service Management systems.

2.2 Multi-Agent AI and LLM-Based Decision-Making Systems

In recent years, the realm of Large Language Models and Generative AI has spurred researchers to investigate multi-agent systems for tackling intricate reasoning and workflow automation challenges. Tasks are broken down into specialized intelligent components, which enhance multi-agent systems in terms of modularity, interpretability and distributed reasoning. Chuanneng Sun, Songjun Huang, and Dario Pompili [4] presented a comprehensive study on LLM-based multi-agent decision-making systems. They studied the importance of reasoning, shared state space, inter-agent sequential communication, and collaborative planning within LLM pipelines. The study highlighted the importance of developing agentic AI architectures in

enterprise automation. The work however, did not gave results for deterministic sequential workflows and feed-forward execution between specialised agents. David Tipe-Palomino and Wilver Auccahuasi [5] developed a helpdesk chatbot using NLP and deep learning techniques for incident classification and first-level issue resolution. The system has been able to automate the incident handling and response generation process. The chatbot, however, was not designed to include multi-dimensional reasoning tasks like priority prediction, queue routing, or decomposition of tasks into modules by different agents. S. Anish, B. N. Rahul, K. R. Pavan Kumar, R. Sankar, and B. M. Mala [6] proposed a customer support ticket generation framework integrating Robotic Process Automation (RPA) with Large Language Models. Their work employed few-shot prompting and chain-of-thought reasoning in the context of extracting tickets, ticket classification, sentiment analysis, and SLA management. While the study showed the effectiveness of prompt engineering with LLMs, the framework did not explicitly decompose the process into agent-level actions, nor did it explicitly transfer state between the stages of the reasoning pipeline. The results of these studies demonstrate increased workflow modularity and reasoning capabilities of the multi-agent AI systems; however, they still have non-deterministic state passing, specialized sequential execution and do not have a structured decision making across multiple attributes of a ticket.

2.3 Prompt Engineering, Agent Architectures, and Retrieval-Augmented Systems

Prompt engineering and Retrieval-Augmented Generation (RAG) techniques have become important components in modern LLM-based intelligent systems. There have been several research interests in enhancing LLM performance in real-world tasks, such as semantic retrieval, modular prompting, and contextual reasoning. Vijay Raj P and Jyoti Shetty [7] presented a comprehensive survey on Large Language Model-based agents for educational systems. They explored the concept of modular agent

architectures with perception, memory, reasoning, and action layers, along with prompt engineering strategies in their study. However, they didn't evaluate open source LLM combinations under resource limited environments. Chathurya Adapa et al [8]. introduced a multi-agent AI framework for agile workflow automation and issue resolution. They used autonomous agents for task extraction and blocker analysis for workflow management systems. The framework was able to effectively coordinate agents, but the underlying reasoning process was more about the workflow than about the rich and natural language input in the tickets. Sungwook Yoon [9] introduced a theoretical model of LLM-based multi-agent coordination with linguistic and contextual constraints. The study showed that the impact of context-window constraints and semantic ambiguity on the performance of multi-agent coordination is very significant. In terms of retrieval-enhanced prompting and semantic example injection mechanisms to enhance downstream agent decisions, however, the work did not cover the same. Lakshan S. N et al [10] introduced a multimodal, memory-based, and Retrieval-Augmented Generation (RAG) Generative AI (GAI) based chatbot for customer IT support automation. The proposed framework highlighted conversational intelligence and interaction in context. The study, however, was more about conversational AI behaviors and lacked details of explicit decision traceability and structured state passing between independent agents. The results suggest that the contextual reasoning performance of LLM systems can be greatly enhanced using prompt engineering and retrieval-based methods. But there is a need for structured workflows involving multiple agents that can interoperate through the semantic retrieval, explainable reasoning and deterministic execution for intelligent ticket automation.

2.4 Research on Prompt Optimization and LLM Workflow Efficiency

Prompt engineering plays a critical role in improving the performance, accuracy, and reasoning capabilities of Large Language Models. Several studies have investigated

prompt optimization techniques for enhancing instruction-following and contextual understanding. Daeseung Park, Gi-taek An, Chayapol Kamyod, and Cheong Ghil Kim [11]. studied the impact of prompt engineering on generative AI systems using Large Language Models. Their study shows that structured prompts, queries, and optimizing instructions increases accuracy, and semantic similarity of the responses generated. The study highlighted the significance of crafting effective prompts to ensure consistent LLM performance. The work mainly targeted single-stage prompting, however, and failed to account for optimization strategies of prompts in multi-agent workflows with outputs that depend on the specific task, and sequential transfer of states between agents. These studies offer insights into the critical role of engineering in intelligent automation systems.

2.5 Emerging Multi-Agent LLM Frameworks for Intelligent Automation

Recent studies have increasingly turned to multi-agent architectures and intelligent automation with the help of Large Language Models and complex reasoning tasks. Shivansh Pandey, Sandhya Avasthi, and Aditya Tandan [12]. proposed a conversational career guidance system using LangChain and an agentic AI framework. They used autonomous conversational agents and multi-step reasoning in their system to deliver personalised career recommendations and skill gap analysis. The framework, however, did not consider structured ticket automation workflow or semantic queue routing, but rather was predominantly for conversational recommendation systems. Johny Gemayel and Ali Mokh [13] implemented a multi-agent orchestration framework for telecom network function automation using LLM. They worked with Semantic Router, Intent Translator, and Resolver agents that operate in a chain-of-thought fashion. The framework showed good orchestration capabilities but mainly on telecom administration workflows, without customer support ticket processing and Retrieval-Augmented Generation integration. Aneta

Poniszewska-Marańda, Maciej Kopa, and Bożena Borowska [14] explored multi-agent systems combined with Large Language Models for intelligent information retrieval. Their work clearly showed that the autonomous agents working together improved retrieval precision and understanding of context. The framework, however, primarily addressed information retrieval tasks, and was not capable of supporting multi-stage reasoning when it came to enterprise ticket automation.

Arjun Sudheer, Chia-Hong Chou, and Shubham Kumar [15] proposed a real-time multi-agent network attack detection and incident response system using LLMs and RAG techniques. The automated detection and response recommendation tasks were completed successfully with the help of special agents in the framework. However, the project was limited to cyber security applications and not customer support ticket classification or semantic queue routing. Vishal Bharadwaj Meruga [16] proposed a multi-agent collaboration approach for automatic complex tasks based on intent interpretation, task decomposition, and adaptive agent coordination through LLM guidance. The framework showed explainable automation features but did not offer state passing workflows, nor any ticket automation pipelines. The studies showed that multi-agent AI systems have a significant impact on the capability of reasoning and automation. Current research, however, still lacks integrated frameworks that integrate semantic retrieval, Retrieval-Augmented Generation (RAG), sequential state transfer, and lightweight open-source LLM deployment into an end-to-end intelligent customer support ticket automation.

CHAPTER – 3

SYSTEM SPECIFICATIONS

3.1 Software Requirements

The proposed multi-agent Generative AI framework for intelligent customer support ticket automation was developed using several software tools, frameworks, libraries, and deployment technologies. Together, they enable Large Language Model inference, semantic retrieval, orchestration of multiple agents, API communication, visualization for the front-end, database management and containerized deployment. The complete software environment was configured to ensure efficient execution of the LangGraph-based AI workflow and Retrieval-Augmented Generation (RAG) pipeline

3.1.1 Development and Programming Tools

The whole system has been implemented using the main programming language Python 3.10. The decision to choose Python was driven by its rich ecosystem of libraries and tools for machine learning, Natural Language Processing (NLP), Generative AI, API development, and database integration. Visual Studio Code (VS Code) was adopted as the Integrated Development Environment (IDE) for writing, debugging, testing and integrating projects. The development and implementation of the project has been mainly carried out with Windows operating systems. The framework can also be run at production scale on Linux-based environments, as well as be containerized.

3.1.2 AI Frameworks and Model Integration

The proposed system utilizes LangChain and LangGraph to implement the multi-agent AI workflow architecture. LangChain has been applied to prompt engineering, LLM chaining, integrating with semantic retrieval, and pipelines for RAG.

LangGraph was used to implement deterministic sequential workflows, shared state management, and coordination between multiple AI agents. The framework uses open-source quantized Large Language Models (LLMs) like Mistral 7B, LLaMA 2-7B-Chat, and Phi-3 Mini 4K Instruct to support tasks in intelligent ticket processing, including ticket classification, priority prediction, routing to queues, and automated response generation. To minimize computational requirements and to provide efficient local inference without the need for large scale GPU infrastructure, quantized Q4 models were chosen. Semantic similarity retrieval was achieved by embedding Sentence Transformers and FAISS vector database. Ticket texts were embedded using a sentence embedding model to semantically represent the relationships between support tickets, and efficient vector similarity search was conducted using FAISS for both queue prediction and response generation

3.1.3 Backend, Frontend, and Database Tools

The backend REST API system was built using FastAPI for seamless real-time ticket prediction and interaction between the frontend interface and the AI workflow pipeline. The backend receives incoming ticket requests, calls the multi-agent system (LangGraph) and keeps track of various ticket histories, returning structured JSON responses. A user-friendly web interface was created using Streamlit that allows users to submit tickets, visualize the AI predictions, and view the generated response. Real-time interaction with the multi-agent AI pipeline is achieved via the Streamlit interface, using REST API communication. SQLite was the database management system chosen for the storage of ticket information such as Ticket subject, description, predicted ticket categories, priority, assigned queues, generated responses, and timestamp information. SQLite was chosen due to its light weight, efficiency, and ability to use in a local deployment environment. Docker was used to containerize the Streamlit frontend application. The use of containers allowed isolation of dependencies, portability, consistency of environments and scalability across systems.

CHAPTER - 4

SYSTEM DESIGN

4.1 Introduction

The proposed system architecture is a multi-agent Generative AI system for intelligent customer support ticket automation. LLMs, semantic retrieval, RAG, LangGraph workflow orchestration, REST API communication, and interactive visualization in the front end are integrated into an end-to-end IT Service Management (ITSM) system. The framework is intended to be a deterministic sequential workflow with multiple AI agents performing specific reasoning tasks to serve customer support tickets. The architecture does not rely on a single centralized monolithic approach but it breaks down the ticket processing in several intelligent steps: preprocessing, issue classification, priority prediction, queue routing, and response generation.

The modular design ensures better ease of interpretation, scalability, transparency of workflows and consistency of decisions. The architecture also incorporates semantic vector retrieval with FAISS databases to allow for more context-specific understanding and routing through RAG. The backend services are built with FastAPI, the interaction layer is built with Streamlit, and the entire application is containerized using Docker. The overall system architecture also allows for the intelligent processing of tickets in real time through the use of locally deployed open-source LLM, without the need for cloud-based inference infrastructure.

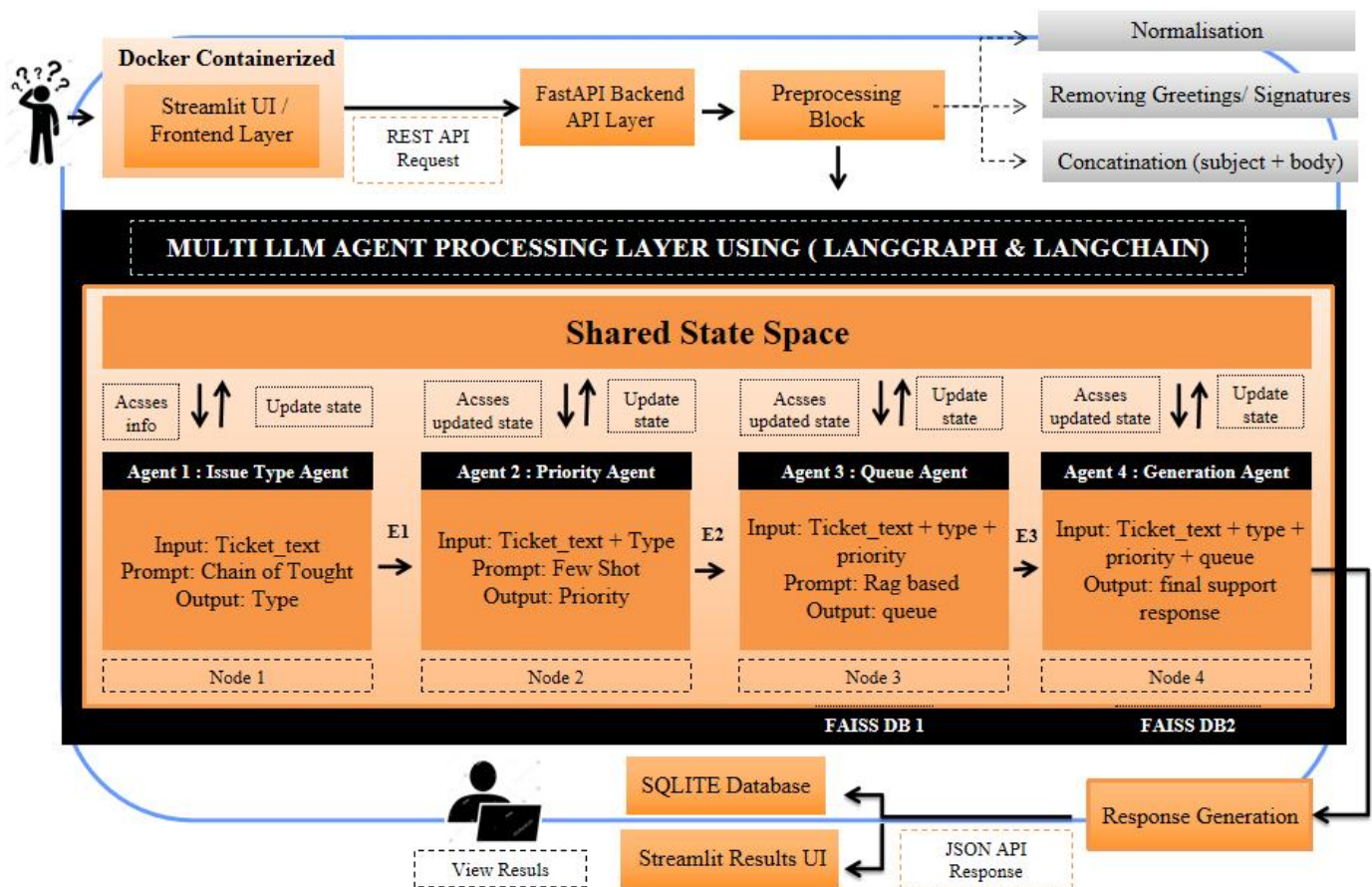


Fig 4.1: The Hybrid RAG-Based Legal Question Answering Framework System Architecture.

4.2 Overall System Workflow

The proposed architecture in Fig 4.1 is a sequential feed-forward workflow beginning from the submission of the ticket and leading to the generation of intelligent response and visualization of the results. The entire process involves the interaction with the frontend, the processing of the APIs on the backend, the preprocessing of the data, the multi-agent reasoning phase, the semantic retrieval phase, the generation of the response, the storage in the database, and the display of the response on the frontend.

The user first files a customer support ticket via the frontend interface, which is implemented with Streamlit. The ticket has a subject and description that describe the customer's issue or request. The Frontend interacts with the Backend using REST API calls built with FastAPI. After receiving the ticket in the backend, the textual content is normalized and cleaned from the ticket. The cleaned ticket is then passed into the LangGraph-based multi-agent workflow system.

The architecture sequentially processes four specialized AI agents, namely issue type prediction, priority estimation, queue routing and contextual response generation. Inbetween predictions and information are stored in a shared state space which is updated by each agent. The new state is automatically propagated to downstream agents via LangGraph workflow edges. The queue routing and the generation of responses are enabled by the integration of semantic retrieval in FAISS vector databases to fetch relevant examples of tickets. Finally, the created results are saved in the SQLite database and returned in structured JSON format to the front end interface. As the ticket is generated, the predicted category, priority, routing queue and generated support response are displayed in real-time to the user on the frontend.

4.3 Preprocessing and Input Processing Layer

The preprocessing layer as depicted in Fig 4.1 is used to preprocess raw customer support tickets before they are passed to the multi-agent reasoning pipeline. Customer tickets frequently have inconsistent formatting, greetings, signatures, URLs, and excessive spaces and noisy text that can have a detrimental impact on semantic understanding and prediction of accurate answers. Ticket subject will be combined with the ticket body into single ticket text representation so that the AI agents can access the full context of the issue described at once, significantly enhancing their contextual understanding. These operations include converting text into lowercase format, removing URLs, eliminating greeting phrases such as “Hello” or “Dear

Support Team,” removing signatures such as “Thanks” and “Regards,”. The preprocessing layer makes sure that semantically relevant information is maintained and that irrelevant text noise is discarded. As shown in Fig 4.1 the cleaned ticket text is then moved to LangGraph orchestrated multi-agent pipeline for decision making. This pre-processing step ensures semantic consistency, better quality embedding during vector retrieval and more reliable predictions for downstream LLMs.

4.4 Multi-Agent Processing Architecture

The key element of the proposed architecture is the multi-agent processing architecture that leverages LangGraph and LangChain. The architecture splits the ticket processing into 4 independent AI agents, each performing a special reasoning task. The proposed architecture addresses this by breaking down decision-making into modular reasoning stages, which are executed in separate prompts. This makes the workflow more explainable, modular, scalable and deterministic

The multi-agent workflow contains the following specialized agents:

- Issue Type Agent
- Priority Prediction Agent
- Queue Routing Agent
- Response Generation Agent

Every agent executes in turn, each having their own access to the shared state space controlled by LangGraph. Shared state structure keeps ticket information and intermediate predictions which are produced in the execution of the workflow. The workflow starts with the Issue Type Agent that forecasts the category of the ticket (Incident, Problem, Request or Change). The predicted category is placed in the shared state and passed on to the next agent. The Priority Agent gets the ticket

content after it has been cleaned and predicts the type of problem and suggests how urgent the ticket is. This is then added to the shared state structure as the expected priority. The Queue Routing Agent gets the ticket text, issue type and priority level. This agent uses Retrieval-Augmented Generation and semantic retrieval techniques to identify the most appropriate support queue for handling the ticket. Finally, the Response Generation Agent receives a complete shared state that includes ticket text, issue category, priority level and assigned queue. With semantic retrieval and prompt engineering, the agent generates professional response. .

4.5 Shared State Management and LangGraph Orchestration

The proposed architecture aims to enhance LangGraph and Langchain for sequential execution of agents. LangGraph supports deterministic state based orchestration, in which the intermediate outputs of one agent can be automatically passed down to downstream agents. Tickets are stored in a shared state structure, TicketState, which is maintained during the running of the workflow. This shared state contains important ticket attributes including:

- Ticket text
- Predicted issue type
- Predicted priority
- Assigned queue
- Generated response

Each AI agent accesses the current workflow state, performs its reasoning task, updates the state with new predictions, and passes the updated state to the next workflow node. LangGraph workflow edges set the order of agents' executions. The architecture is based on a fixed sequence of agents, such that the output of one agent constitutes the input to the next agent's reasoning task. This state passing mechanism

helps to achieve consistency of predictions and enables downstream agents to benefit from past reasoning decisions. Moreover, the modularity and transparency of the workflow are enhanced by the way each reasoning task is encapsulated in its own execution node with LangGraph. This allows for simpler debugging, scalability, replacing models and future expansion of workflows.

4.6 Semantic Retrieval and Response Generation

The framework proposed in this paper combines semantic retrieval and Retrieval-Augmented Generation techniques to enhance the understanding and prediction of context when handling queue routing and generating responses. FAISS vector databases and Sentence Transformer embeddings are used to implement semantic similarity search. Tickets are converted to dense vector representations and stored in FAISS dense vector indexes. During inference, semantically similar tickets are retrieved based on cosine similarity between embeddings. Before queue prediction, the Queue Routing Agent is able to add semantically similar routing examples to the prompt. This allows the agent to reason about the context instead of fixed classification rules. Likewise, the Response Generation Agent fetches semantically similar historic responses from the vector database. Examples retrieved are used as few-shot prompts to steer the LLM to produce professional, contextually relevant, and structurally consistent responses. The integration of the vector retrieval also decreases hallucination and increases contextual grounding during LLM inference.

4.7 Backend Communication, Database Integration, and Frontend Visualization

This architecture combines backend, database and response display on the frontend within single deployment framework. The backend API layer is implemented using FastAPI, where it receives ticket requests, calls the LangGraph workflow, processes the data to perform preprocessing steps, and outputs structured predictions. The REST API endpoints allow real-time communication between the front and back end. The architecture also features SQLite database management, which takes care of ticket history and ensures it remains persistent. Ticket subjects, descriptions, predicted categories, priority levels, assigned queues, responses generated, and timestamp information are stored in the database. Streamlit is used for the frontend interaction layer. The frontend brings interface for query submission and for visualizing LLM generated outputs in real-time Frontend app is packaged and deployed as a Docker container in its own execution environment. Containerization allows system portability, scalability, dependency and deployment. Combining the backend APIs, semantic retrieval, multi-agent orchestration, database storage, and the visualization on the front-end creates an end to end intelligent IT Service Management architecture ready for real-world deployment.

CHAPTER – 5

SYSTEM IMPLEMENTATION

5.1 Modules used with description:

The proposed multi-agent Generative AI framework for customer support ticket automation was implemented using multiple interconnected modules. Every module carries out a specific task in the workflow pipeline. The implementation integrates them all along the way from the pre-processing to the LLM inference, semantic retrieval, multi-agent orchestration, backend communication, database integration, and the visualization to the frontend, making it an end-to-end intelligent IT Service Management system.

The system implementation is broken down into the following major modules:

5.1.1 Input and Preprocessing Module

The Input and Preprocessing Module is in charge of collecting raw customer support tickets and pre-processing them for intelligent processing in the multi-agent workflow pipeline. Greetings, signatures, URL, newline characters and extraneous text noise found in customer tickets tend to be inconsistent and can have a detrimental impact on semantic understanding and the accuracy of prediction.

This module gets the ticket subject and ticket body from the frontend interface. Both these fields are joined to create a single representation of the ticket text. Combining subject and description information improves contextual understanding because the AI agents can analyze the complete issue description together. Several Preprocessing Steps are done after concatenation. The preprocessing functions include converting text into lowercase format, removing URLs, eliminating greeting phrases such as “Hello,” “Dear Team,” and “Good Morning,” removing signatures such as “Thanks,”

“Regards,” and “Sincerely,” and removing newline characters and extra spaces. The preprocessing module is used to normalize the ticket text while retaining semantically important information that is useful for downstream reasoning tasks. The cleaned text of the ticket is then passed on to the multi-agent workflow pipeline, where classification, routing, and generating responses are performed. This module enhances the semantic consistency, embedding quality, and prediction accuracy of the proposed system.

5.1.2 Issue Type Classification Module

The Issue Type Classification Module is responsible for predicting the category of the customer support ticket. This module is part of the first reasoning phase of the multi-agent workflow architecture in LangGraph. The module receives the cleaned ticket text from the preprocessing layer and uses a prompt-engineered LLM to classify the ticket into one of the predefined categories:

1. Incident
2. Problem
3. Request
4. Change

The classification process is based on prompt-based reasoning without the use of the traditional supervised classification models. A structured prompt with category definitions, decision rules, and classification instructions is given to the LLM, to get its prediction behavior.

The model uses ticket semantics, ticket descriptions, service disruptions and context to determine which category the ticket belongs to. LangGraph manages the shared workflow state by adding the predicted issue type to it. This output is used as a contextual input to the next modules, e.g., priority prediction, queue routing. The

module enhances the explainability of workflows by separating ticket categorization from a centralised AI workflow.

5.1.3 Priority Prediction Module

The Priority Prediction Module estimates the urgency and severity of the customer support ticket. This step is the second reason in the multi-agent workflow. The cleaned ticket text and the predicted ticket type are passed to the module from the previous stage in the workflow. Priority estimation process is based on promotion, business impact, service disruption and operational necessity.

The module predicts one of the following priority levels:

1. High
2. Medium
3. Low

The Large Language Model is prompted in a chain-of-thought style, leading it to give a structured answer prior to prediction. To ensure consistency of prediction, the prompts include severity definitions, operational examples, workflow rules and decision instructions. Before assigning a priority level, the module judges whether this issue is affecting business continuity, preventing access to the system, poses a security risk, or causes degradation of workflow. LangGraph adds the predicted priority to the shared state structure which is responsible for maintaining and that is passed to the downstream agents for the purpose of giving them instructions about routing and generating the response. This module facilitates structured urgency estimation in the proposed IT Service Management framework, while enhancing intelligent ticket handling.

5.1.4 Queue Routing Module

The Queue Routing Module is responsible for assigning customer support tickets to the most appropriate support department or service queue. This module is one of the most crucial reasoning modules as it has a direct impact on the efficiency of ticket resolution, so the accuracy of the routing matters here. Previous stages of the workflow provide the cleaned ticket text, expected issue type and expected priority level to the module. The proposed framework incorporates Retrieval-Augmented Generation and semantic similarity retrieval in contrast to traditional keyword-based routing systems, to enable context-aware queue prediction. Historical ticket examples are embedded and stored in FAISS vector databases as sentence embeddings. semantically similar ticket examples are retrieved using vector similarity search. The retrieved examples are fed into the routing prompt to give contextual guidance to the Large Language Model.

The module predicts one of several predefined support queues including:

1. Technical Support
2. Product Support
3. Billing and Payments
4. Customer Service
5. IT Support
6. Service Outages and Maintenance
7. Human Resources
8. Sales and Pre-Sales
9. Returns and Exchanges
10. General Inquiry

The Queue routing is achieved using semantic reasoning rather than exact keyword match. This will enhance precision routing for unsighted variations of tickets and context-rich support tickets. The queue predicted is stored in the shared state space which is then passed to Response Generation Module.

5.1.5 Response Generation Module

The Response Generation Module generates professional, context-aware customer support responses based on the information in the tickets, and the predictions in the workflow.

This module receives the complete workflow state which contains:

- Ticket text, Predicted issue type, Predicted priority, Assigned support queue

This module combines Retrieval-Augmented Generation with FAISS vector retrieval and semantic similarity search. Historical ticket responses are mapped to the vector space and stored in semantic vector databases. Historical ticket responses are embedded in vectors and stored in semantic vector databases. During retrieval, semantically similar responses are retrieved and given to the prompt as few-shot examples. This Prompt will then given to Large Language Model for generating appropriate support responses. The response generation process is performed through open-source Large Language Models, like Mistral and LLaMA 2, with low-temperature inference to minimize randomness and enhance the stability of the generated responses. The generated response is then merged with the other state in the workflow and returned to the frontend to be displayed. This module facilitates intelligent automated assistance to customers in the proposed framework.

5.1.6 LangGraph Workflow Orchestration Module

The LangGraph Workflow Orchestration Module handles the coordination of all AI agents and ensures the execution of the structured workflow throughout the system. The module adopts a deterministic sequential workflow architecture – each reasoning stage is modeled as a separate workflow node. LangGraph orchestrates the passing of states between nodes and manages order of the multi-agent pipeline.

Shared state is stored in a structure referred to TicketState as it is used during the execution of the workflow. The shared state stores:

- Ticket text
- Predicted issue type
- Predicted priority
- Assigned queue
- Generated response

Every agent looks at the current state, carries out the reasoning process, changes the state of the workflow, and passes the new state to downstream agents.

Workflow edges define the execution order between nodes:

Type Agent → Priority Agent → Queue Agent → Response Agent

This deterministic feed-forward workflow prevents uncontrolled branching, enhances workflow transparency, explainability and modularity.

5.1.7 FastAPI Backend Module

The FastAPI Backend Module provides real-time communication between the frontend interface and the multi-agent AI workflow pipeline.

The backend exposes REST API endpoints for:

- Ticket prediction
- Ticket history retrieval
- Backend status monitoring

When User submits a query from frontend interface, which triggers the LangGraph workflow pipeline, which processes the ticket in various ways, stores the processed data, and produces structured responses in JSON format. The backend also adds exception handling and traceback logging, providing better reliability and debugging assistance. FastAPI was selected because of its high performance, asynchronous request handling, and efficient API deployment capabilities..

5.1.8 SQLite Database Module

The SQLite Database Module handles ticket history and persistent records of support tickets processed. If the history table is not created, the module automatically initializes the database and creates the table when the system loads.

The database stores:

- Ticket subject
- Ticket description
- Predicted category
- Priority level

- Assigned queue
- Generated response
- Timestamp information

The module provides database operations for:

- Table initialization
- Ticket insertion
- Ticket retrieval

SQLite was chosen for its light-weight, easy to integrate, and local deployment capabilities. This database module highlights ticket logging, workflow monitoring, and frontend history visualization.

5.1.9 Frontend User Interface Module

The Frontend User Interface Module was implemented using Streamlit to provide real-time interaction with the multi-agent AI system.

- The interface allows users to:
- Submit customer support tickets
- Enter ticket subject and description
- View predicted ticket type
- View predicted priority
- View assigned queue
- Display generated support response

The frontend communicates with the backend using REST API requests. To enable interactive workflows of users and real-time visualization of results, dynamic session state management was implemented. The interface also features cinematic UI styling, background effects, glassmorphism elements, and interactive animations to enhance user experience. This module enables user-friendly interaction with the intelligent ticket automation framework.

5.1.10 Docker Deployment Module

The Docker Deployment Module is used to provide containerized deployment support for the frontend application.

The Streamlit frontend application was containerized using Docker to achieve:

- Dependency isolation
- Environment consistency
- Portable deployment
- Scalable execution

Docker allows the application to be run with the same consistency across various systems without any dependency issues. The frontend container uses REST API endpoints to communicate with the FastAPI backend container. Containerization makes it easier to maintain and deploy for a production-level execution environment.

5.2 Implementation Workflow

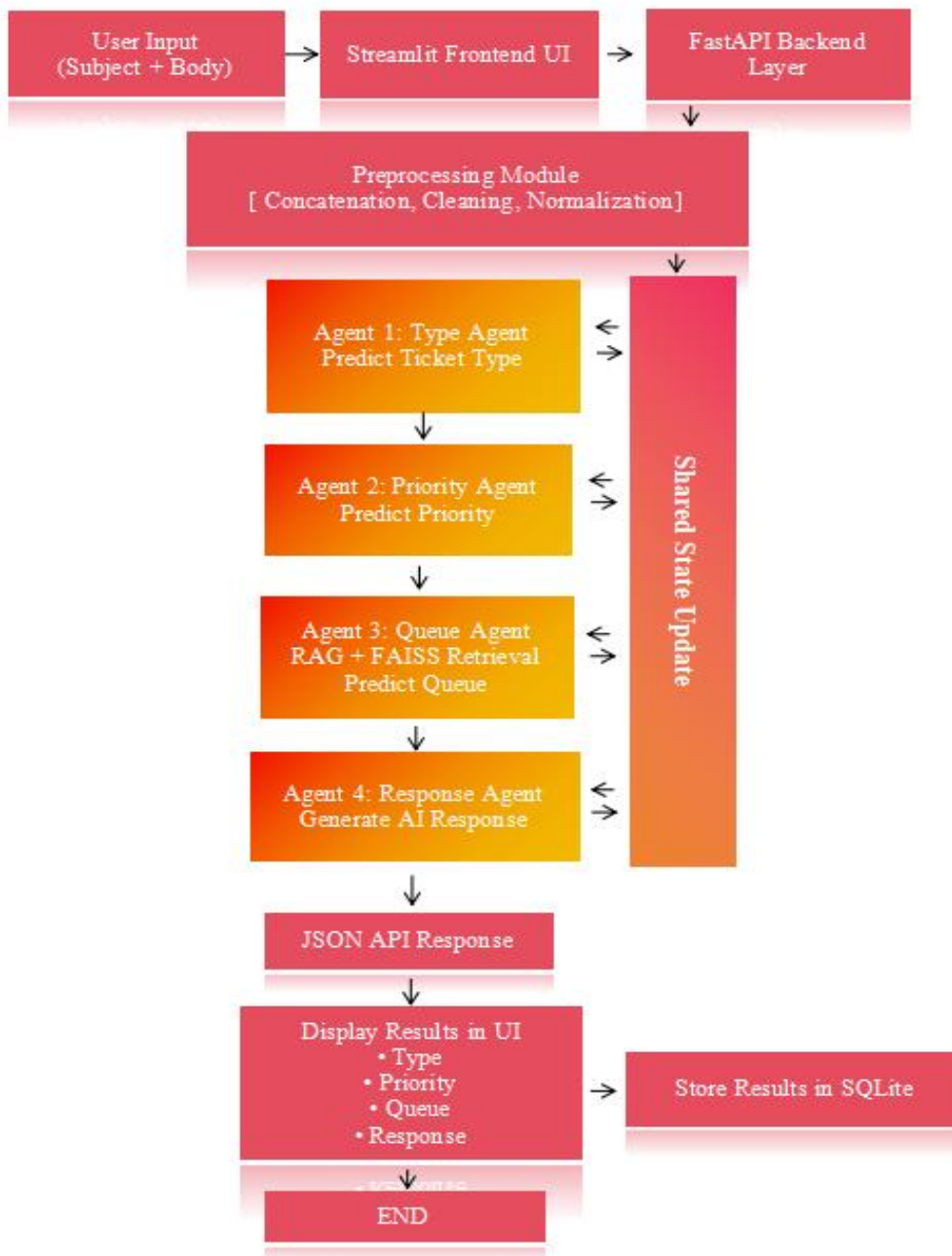


Fig 5.1 Implementation Workflow

The overall workflow of the proposed multi-agent customer support ticket automation system is illustrated in Figure 5.1.

The system starts with the user inputs that is presented in the frontend using Streamlit containing the ticket subject and the ticket body. The frontend is connected to the backend via REST API requests with FastAPI. The request is received and forwarded to the preprocessing module where concatenation, cleaning and normalization operations are performed on the ticket. The preprocessing stage unites the subject and body into a single ticket representation and eliminates irrelevant textual components like greetings, signatures, URLs and extra spaces for improved semantic consistency. The cleaned text is then fed into the LangGraph based multi-agent processing workflow. The first agent is the Type Agent which predicts the ticket type (Incident, Problem, Request, Change). The output predicted will be written to the shared state structure and passed to the next stage of the workflow.

The second agent, called the Priority Agent, reads the content of the ticket and the anticipated type of issue to determine the priority of the ticket. The predicted priority is once again updated in the shared state space. The third agent, called the Queue Agent, performs queue routing using Retrieval-Augmented Generation (RAG) and FAISS-based semantic retrieval. This agent provides the most relevant support queue according to context understanding of the tickets and semantic examples retrieved. The fourth and final agent is the "Response Generation Agent" which creates a context-aware AI response for support from the ticket text, predicted ticket type, predicted ticket priority and assigned queue. Once the multi-agent workflow execution is done, the generated outputs are returned as structured JSON API response. These results are presented in the frontend interface of Streamlit, along with the predicted ticket type, priority, assigned queue, and generated support response. Concurrently, processed ticket information and the generated outputs are stored in the SQLite database for ticket history management and retrieval in the future. In this way, the proposed workflow can be used for intelligently handling end-to-end customer support ticket automation using sequential multi-agent reasoning, semantic retrieval, and real-time frontend-backend integration, as illustrated in Figure 5.1.

CHAPTER - 6

SYSTEM TESTING

The proposed multi-agent customer support ticket automation framework was tested with classification-based metrics and semantic response generation metrics. Given multiple reasoning tasks performed by the system: ticket type classification, priority prediction, queue routing, and contextual response generation, separate evaluation strategies were used for the different tasks. The evaluation process aimed to assess the accuracy of the predictions, the ability to understand the semantics, the quality of the responses, and the overall reliability of the workflow for various combinations of LLMs. A series of LLaMA 2, Mistral, and Phi-3 models were assessed using four different configurations: Type Agent, Priority Agent, Queue Agent, and Response Generation Agent to determine the best balance of models for the architecture.

The performance evaluation was divided into two major categories:

- Classification Task Evaluation
- Response Generation Evaluation

6.1 Performance Metrics for Classification Tasks

The system was evaluated using nine different LLM combinations constructed using Mistral, LLaMA 2, and Phi-3 models across different workflow agents. The predicted output produced by each agent was compared to the respective ground-truth labels found in the customer support ticket data set. Separate evaluations were performed for: Ticket Type Prediction, Priority Prediction, Queue Routing Prediction. In order to gain insight into reasoning capability and prediction consistency at the agent level, the classification performance was analyzed separately for each stage of the

workflow. Several standard supervised learning evaluation metrics were adopted for assessing the classification performance such as accuracy, precision, recall, F1-Score, Macro F1-Score, and Weighted F1-Score. Further, a classification report was created for each workflow agent using the `classification_report` function in Scikit-learn:

6.1.1 Accuracy

Accuracy is the percentage of total samples that are correctly classified. It gives an overall approximation of how correct the model is in predicting all classes.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \quad (1)$$

Where:

TP = True Positives

TN = True Negatives

FP = False Positives

FN = False Negatives

The higher the accuracy value, the better the overall classification.

6.1.2 Precision

Precision is the ratio of true positives to all positive predictions made by the model. When misclassification of positive predictions is costly in workflow routing systems, precision is important. The higher the precision, the more correct predictions

$$Precision = \frac{TP}{TP+FP} \quad (2)$$

6.1.3 Recall

Recall is the capability of the model to correctly classify genuine positive samples from the data set. Recall becomes important when missing important ticket categories or urgent issues can negatively affect support workflows.

$$Recall = \frac{TP}{TP+FN} \quad (3)$$

The higher the recall value, the better the system is able to detect a class.

6.1.4 F1-Score

F1-Score is the harmonic mean of Precision and Recall. It offers fair evaluation when the data set is imbalanced or has non-uniform label distribution

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (4)$$

The F1-score was especially significant in assessing the performance of the queue routing as some labels on the queues were less frequent than the others in the data set.

6.2 Response Generation Evaluation

The Response Generation Agent has been assessed separately as response generation is a natural language generation task, not a classification task. The responses created were then compared with support responses which was included in the dataset. The evaluation process focused on measuring:

- Semantic similarity
- Contextual alignment
- Response structure
- Instruction following

Response evaluation included quantitative assessment measures and qualitative inspection measures.

6.2.1 ROUGE-L Evaluation

ROUGE-L was used to measure overlap-based similarity between generated responses and human-written responses.

The ROUGE-L metric measures:

- Structural similarity
- Lexical overlap
- Response alignment

ROUGE-L scores get higher with greater overlap between the generated and reference responses. ROUGE-L was helpful in assessing the consistency of information in the human-written support responses that is retained in the generated responses.

6.2.2 BERTScore Evaluation

BERTScore was used to evaluate semantic similarity between generated responses and reference responses using contextual transformer embeddings. The BERTScore process calculates similarity by generating contextual embeddings with transformer models. The semantic similarity between embeddings is measured using cosine similarity:

$$Similarity = \frac{A \cdot B}{||A|| ||B||} \quad (5)$$

Where:

- A represents generated response embeddings
- B represents reference response embeddings
- Higher BERTScore values indicate stronger semantic alignment between generated and human-written responses.

CHAPTER – 7

RESULTS AND ANALYSIS

This chapter presents the experimental results and performance analysis of the proposed multi-agent Generative AI framework for intelligent customer support ticket automation. Multiple combinations of open-source LLMs, such as Mistral 7B, LLaMA 2-7B-Chat, and Phi-3 Mini, were evaluated using the framework with various workflow agents. The assessment centered on quantifying the success of ticket type prediction, priority assessment, queue routing, and AI response generation.

The proposed framework was tested using the “Customer Support Tickets” dataset obtained from Hugging Face. The experiments were performed with nine combinations of models among the Type Agent, Priority Agent and Queue Agent. Semantic similarity metrics were used to evaluate the Response Generation Agent individually.

The following abbreviations were used to represent model combinations:

- M = Mistral 7B
- L = LLaMA 2-7B-Chat
- P = Phi-3 Mini

For example:

- MML indicates Mistral for Type Agent, Mistral for Priority Agent, and LLaMA 2 for Queue Agent.
- MMM means Mistral used in all workflow agents.

The results presented in this chapter demonstrate the effectiveness of multi-agent orchestration, Retrieval-Augmented Generation, semantic retrieval, and mixed-model reasoning for intelligent IT Service Management automation.

7.1 Type Agent Results and Analysis

The Type Agent is responsible for predicting the category of customer support tickets. The ticket categories used in the proposed framework include:

- Change
- Incident
- Problem
- Request

The results of the experiments performed on the Type Agent are presented in Table 7.1 and graphics comparison of model accuracy is presented in Figure 7.1.

Table 7.1 Type Agent Classification Results

Models	Change	Incident	Problem	Request	Accuracy
PML	0.76	0.61	0.56	0.92	0.69
MLP	0.9	0.92	0.83	0.97	0.92
LLL	0.27	0.77	0.0	0.0	0.48
MML	0.91	0.93	0.83	0.97	0.92
PPP	0.79	0.6	0.57	0.91	0.68
MPL	0.91	0.93	0.83	0.97	0.92
PPL	0.76	0.61	0.56	0.92	0.69
MMM	0.89	0.92	0.83	0.97	0.91
LMP	0.29	0.77	0.0	0.0	0.48

The experimental results show that the MML, MLP, and MPL configurations achieved the highest overall accuracy of 0.92, while the MMM configuration

achieved 0.91 accuracy. The results show that mixed-model based architectures perform much better than single-model based architectures for ticket type classification tasks. The Request class had the best prediction performance for most model combinations, with scores as high as 0.97. This means that request-based tickets have more semantic structures and more distinguishable textual patterns than other types of tickets. .

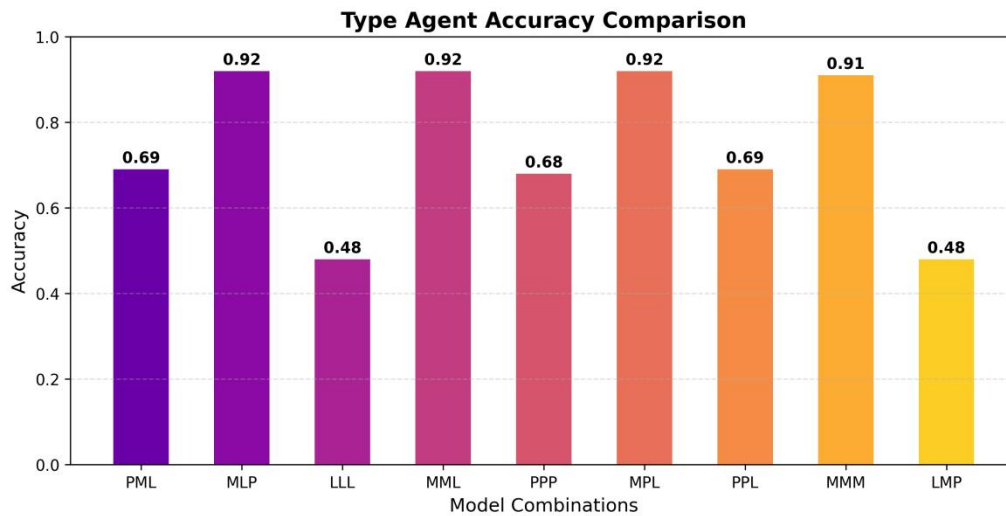


Figure 7.1 Type Agent Accuracy Comparison

Lower performance was seen for the Problem category as problem-related tickets tend to be semantically similar to Incident-related tickets and technical issue descriptions. The LLL and LMP configurations produced the lowest overall performance with only 0.48 accuracy. The results show that pipelines using only LLaMA models are not as consistent as Mistral-based pipelines in determining the classification. It is evident from the graph in Figure 7.1 that the Mistral-integrated configurations are clearly dominating the issue type prediction. The outcomes show that Mistral is more capable of following instructions and understanding the semantic meaning in classification tasks. Overall, the Type Agent performed best across all workflow steps, due to the relatively strong semantic properties of the categories of tickets.

7.2 Priority Agent Results and Analysis

The Priority Agent classifies customer support tickets with semantic understanding and contextual reasoning to predict their urgency level. The system predicts the following priority levels:

- High
- Medium
- Low

The detailed experimental results are presented in Table 7.2, while the graphical comparison of model accuracies is shown in Figure 7.2.

Table 7.2 Priority Agent Classification Results

Models	High	Low	Medium	Accuracy
PML	0.85	0.78	0.72	0.79
MLP	0.67	0.39	0.0	0.52
LLL	0.67	0.41	0.0	0.53
MML	0.91	0.8	0.77	0.84
PPP	0.85	0.72	0.53	0.73
MPL	0.85	0.72	0.53	0.74
PPL	0.85	0.73	0.54	0.74
MMM	0.91	0.8	0.76	0.84
LMP	0.67	0.72	0.6	0.67

Experimental analysis reveals that priority estimation is more difficult than ticket type classification given the constraints of a business impact assessment and the need to determine the severity of the service disruption and the urgency of the operational need, based on free-form text descriptions. The MMM and MML configurations performed best with overall accuracy of 0.84. These setups also yielded the best results for High-priority tickets, with a prediction score of 0.91.

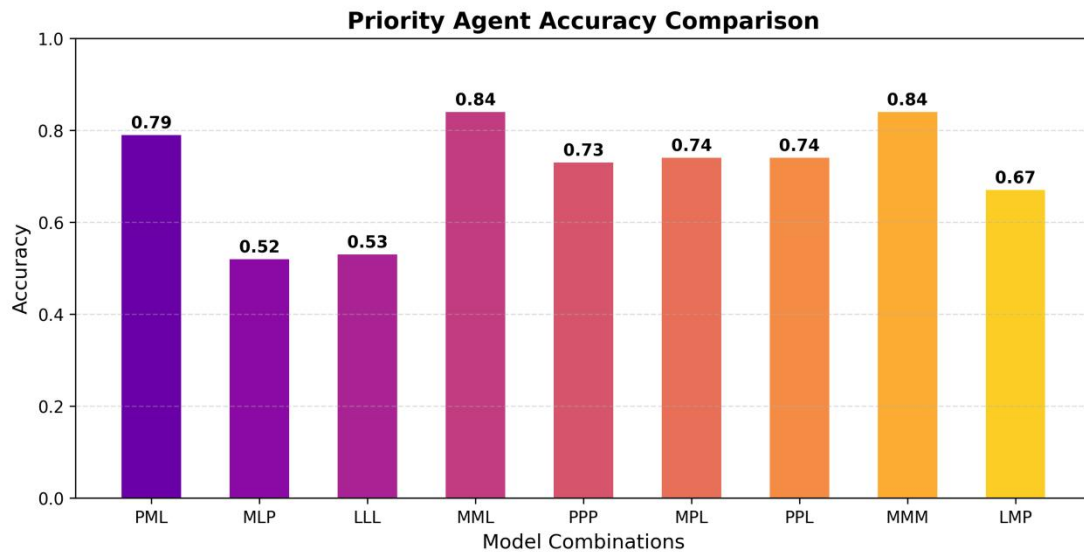


Figure 7.2 Priority Agent Accuracy Comparison

High priority tickets had better prediction performance due to explicit marking of urgency in the tickets such as:

- Service outages
- Payment failures
- Access restrictions
- System crashes
- Security concerns

Medium-priority prediction has comparatively lower performance since the contextual patterns in medium-priority tickets are often ambiguous and are shared by both high and low priority tickets. The MLP and LLL configurations showed worse overall performance results in the range of 0.52-0.53 accuracy, suggesting poorer contextual reasoning abilities when performing severity estimation tasks. Figure 7.2 shows the performance of cross model architectures in priority estimation surpassed the single model pipelines. Chain-of-thought prompting showed superior consistency of logical reasoning for prediction tasks. Overall, the results validate the claim that priority estimation involves more semantic reasoning than type classification..

7.4 Queue Agent Results and Analysis

The Queue Agent performs intelligent queue routing using semantic retrieval and Retrieval-Augmented Generation. The detailed routing results are presented in Table 7.3, while graphical comparison of overall routing accuracies is shown in Figure 7.3.

The queue categories include:

- Billing & Payments
- Customer Service
- General Inquiry
- HR
- IT Support
- Product Support
- Returns and Exchanges
- Service Outages and Maintenance
- Technical Support

Table 7.3 Queue Agent Classification Results

Models	Billing & Payments	Customer Service	General Inquiry	HR	IT Support	Product Support	Returns & Exchange	Sales & Pre-Sales	Service Outages	Technical Support	Accuracy
PML	0.91	0.71	0.67	0.43	0.58	0.67	0.57	0.59	0.66	0.8	0.69
MLP	0.88	0.57	0.0	0.44	0.39	0.49	0.33	0.6	0.55	0.65	0.57
LLL	0.87	0.68	0.48	0.43	0.48	0.63	0.46	0.29	0.62	0.65	0.64
MML	0.86	0.73	0.57	0.43	0.58	0.69	0.57	0.59	0.62	0.79	0.73
PPP	0.89	0.62	0.44	0.6	0.39	0.5	0.46	0.6	0.51	0.64	0.58
MPL	0.88	0.72	0.4	0.43	0.54	0.69	0.62	0.59	0.61	0.79	0.72
PPL	0.89	0.72	0.44	0.43	0.52	0.66	0.62	0.59	0.64	0.79	0.68
MMM	0.96	0.67	0.67	0.77	0.58	0.67	0.67	0.56	0.57	0.77	0.71
LMP	0.88	0.57	0.57	0.4	0.38	0.51	0.46	0.63	0.46	0.64	0.56

Queue routing represented the most difficult workflow task because of:

- Larger number of output classes
- Semantic overlap between categories
- Various Contextual meanings in tickets

The highest overall queue routing accuracy was 0.73, for the MML configuration, followed closely by MPL with 0.72 and MMM with 0.71. The Billing & Payments category performed well under most settings due to the presence of highly unique contextual keywords and financial transaction patterns in billing-related tickets. The other two categories, Technical Support and Product Support, also produced relatively high prediction scores as technical problems typically have explicit operational indicators. The HR and General Inquiry categories had relatively low performance, because the meaning of these categories is general and are related with more than one support domain.

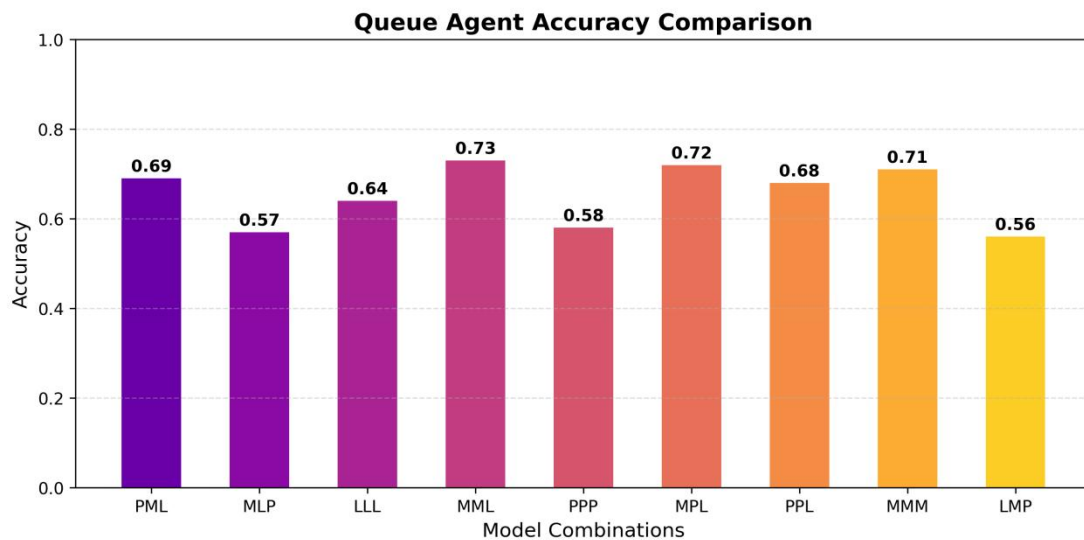


Figure 7.2 Priority Agent Accuracy Comparison

The MMM configuration demonstrated exceptionally strong performance for:

- Billing & Payments: 0.96
- HR: 0.77

These outcomes showed that fully Mistral-based architectures have high contextual understanding capability in special categories of support. Figure 7.3 demonstrates the enhanced routing performance of mixed model pipelines than LLaMA only and Phi only combinations. The novel deployment of semantic retrieval using FAISS and RAG enhanced the uniformity of queue routing by enriching prompts with examples of semantically similar tickets prior to prediction. The overall results indicate that semantic retrieval significantly enhances the understanding and routing accuracy of contextual queues in intelligent IT Service Management systems.

7.4 Response Generation Results and Analysis

The response Agent generates customer support responses using:

- Ticket text
- Predicted ticket type
- Predicted priority
- Assigned support queue

The generated responses were evaluated using:

- BERTScore F1
- Semantic Similarity

Table 7.4 Response Generation Evaluation Results

Combinations	Response Models	BERTScore F1	Semantic Similarity
MMLL	LLaMA 2-7B-Chat (Q4_K_M)	0.89	0.77
MMLM	Mistral 7B (Q4_K_M)	0.9	0.76
MMLP	Phi-3 Mini 4K Instruct (Q4)	0.87	0.74

The response generation results as shown in Table 7.4 shows that the MMLM configuration with Mistral 7B produces the highest BERTScore of 0.90, which indicates good contextual similarity with the human-written support responses. The MMLL configuration with LLaMA recorded the highest semantic similarity score of 0.77, showing high contextual awareness and semantic consistency in the process of generating responses.

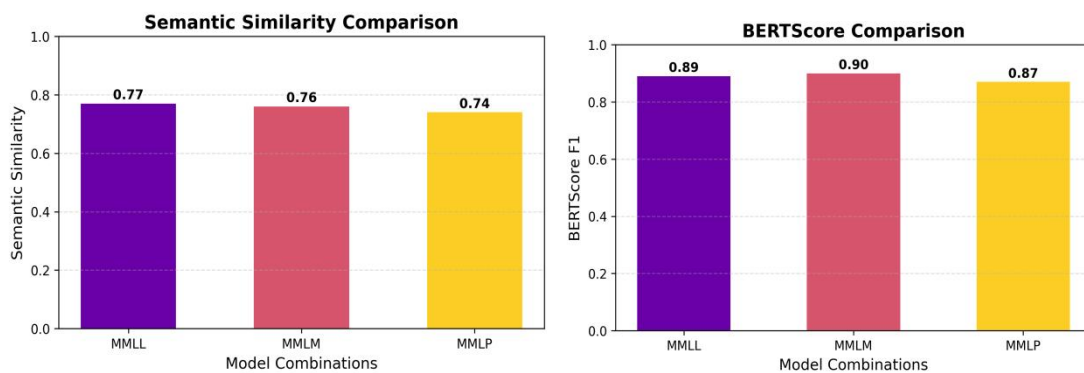


Figure 7.4 BERTScore Comparison & Semantic Similarity of Response Models

The MMLP setup with Phi-3 Mini was found to perform comparatively poorly with::

- BERTScore F1: 0.87
- Semantic Similarity: 0.74

Qualitative evaluation showed that Mistral produced more structured responses with enhanced instruction-following responses than Phi-3 Mini. Retrieval-Augmented Generation boosted response quality by inserting semantically similar historical responses into prompts prior to generation. This decreased hallucinations and increased context awareness. The graphs shown in Figures 7.4 clearly illustrate the superior performance of Mistral-based response generation architectures..

Overall, the findings validate the proposed framework's ability to produce context-relevant support responses that are semantically similar to human-generated answers.

7.5 Comparative Discussion

The overall analysis on experiments shows that, in most workflow tasks, the integrated model architecture is superior to single model pipelines. Best performance stability and balance were attained with mistral-integrated configurations for the multi-agent workflow..

- Several important observations were identified during experimentation:
- Type classification achieved the highest performance among all tasks.
- Priority estimation needs more contextual reference for reasoning
- The incorporation of Retrieval-Augmented Generation has greatly enhanced routing consistency and response quality.
- Mistral outperformed in its ability to follow instructions.
- LLaMA demonstrated strong semantic contextual understanding.
- The response consistency was relatively lower for Phi-3 Mini.

The MML architecture proved to be the most successful workflow configuration across all the evaluated configurations, with better performance in ticket classification, priority estimation and routing tasks. Results of the experiment indicate the effectiveness in enhancing intelligent customer support automation applications as

- LangGraph-based multi-agent orchestration
- Retrieval-Augmented Generation
- Semantic vector retrieval
- Mixed-model AI pipelines
- Prompt-engineered workflow reasoning.

7.9 User Interface and Deployment Results

The developed user interface of the proposed system is shown in Figure 7.6. The frontend was created with Streamlit and connected using FastAPI backend with multi-agent workflow (LangGraph), SQLite database, and Docker deployment framework.

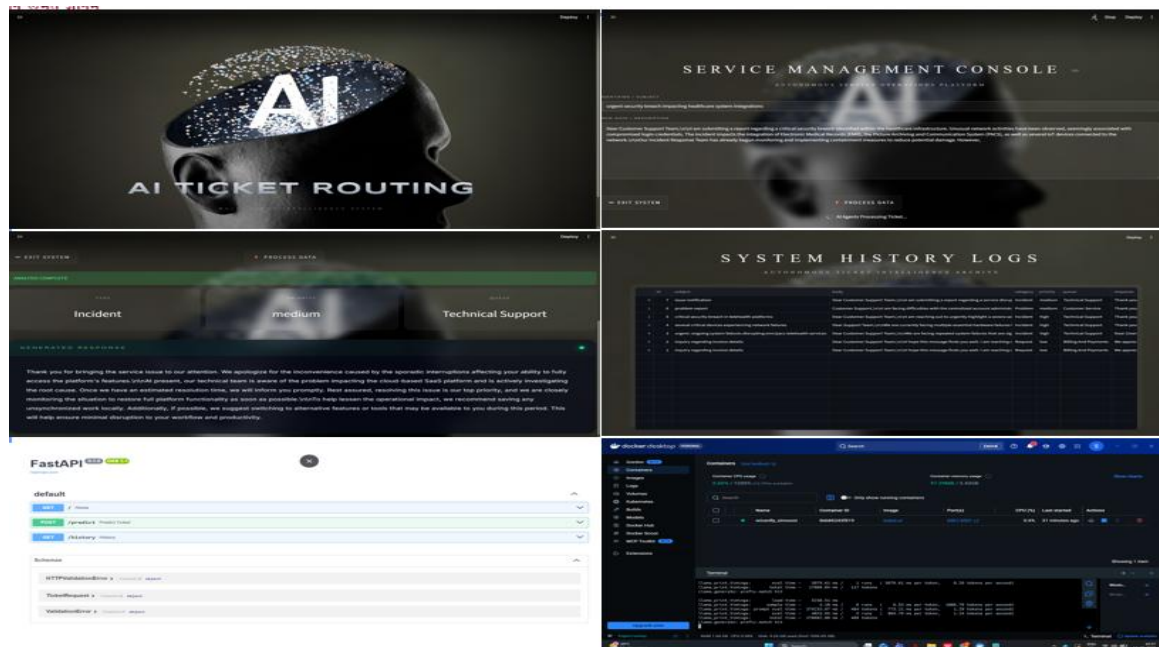


Figure 7.6 User Interface and Deployment Screens

Through UI, users can submit customer support tickets and view agent generated response such as the predicted type of ticket, priority level, assigned queue, and generated support response in real time. There is also a history Log section to view and display any previously processed tickets that have been stored in the SQLite database. The FastAPI interface shows successful deployment of the back-end API in order to predict tickets and retrieve them from history. Docker containerization was used to achieve portable and isolated deployment of the frontend application. Overall, the interface effectively illustrates the seamless interaction between the frontend, backend, multi-agent AI workflow, semantic retrieval system, and database infrastructure in the proposed intelligent customer support automation system..

CHAPTER – 8

CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

The proposed work successfully developed a multi-agent Generative AI framework for intelligent customer support ticket automation using LangGraph and LangChain. The framework adopted several different AI agents for ticket type prediction, priority estimation, semantic routing of queues, and generation of responses using RAG in a sequential process. For intelligent reasoning at various stages of the workflow, the proposed system employed open-source Large Language Models: Mistral 7B, LLaMA 2-7B-Chat, and Phi-3 Mini. The LangGraph state-passing mechanism facilitated multi-agent coordination in the ticket processing pipeline in a structured manner and feed-forward execution deterministically. Experimental evaluation showed that the proposed framework performed well in all the workflow tasks.

The highest overall classification performance was found for the Type Agent, whereas the most difficult task was queue routing due to the semantic similarity between support categories. Overall, the MML architecture (Mistral → Mistral → LLaMA 2) showed the most reliable and stable performance across all evaluated model combinations. The use of FAISS-based semantic retrieval and Retrieval-Augmented Generation enhanced the context understanding, routing consistency, and response quality. The response generation module could generate meaningful responses that are contextually similar to the human-generated answers. The integration of FastAPI, Streamlit, SQLite, and Docker enabled the development of a modular, interactive, and locally deployable intelligent IT Service Management framework. The proposed architecture shows how lightweight open-source Large Language Models can successfully be implemented to automate tickets end-to-end without the need for large-scale cloud infrastructure. In conclusion, the proposed

framework is a successful demonstration of the effectiveness of the multi-agent orchestration, semantic retrieval and Generative AI for intelligent customer support automation in real world IT Service Management environments..

8.2 Future Scope

The proposed framework can be further extended and improved in several directions to support more advanced intelligent customer support automation capabilities. The future development plans include expanding functionality with multilingual ticket understanding and multilingual response generation, which will enable the system to operate in the context of an international business environment and with multilingual customer interactions. Certainly, the system can be expanded with the ability to send uncertain predictions to human support agents for confirmation before final ticket processing using the confidence-based human-in-the-loop validation mechanism. Further enhancements could see the full cloud-native deployment with Docker Compose, Kubernetes and scalable microservice architectures to support enterprise level production environments and to handle large-scale ticket processing. For organization-specific workflows, the ability to fine-tune Domain-Specific Large Language Models can further enhance the accuracy of ticket classification, consistency of queue routing, and quality of the responses generated for tickets. The proposed framework can additionally be integrated with real-time enterprise communication systems including email platforms, CRM systems, and enterprise helpdesk solutions such as ServiceNow, Jira, Zendesk, and Freshdesk. The framework can also be extended with a memory system that adapts to the user's preferences, reinforcement learning for optimizing workflows and cooperative autonomy with agents for complex enterprise IT Service Management automation.

REFERENCES

- [1]. Pereira, Leonardo Santiago Benitez, Robinson Pizzio, Samir Bonho, Leonardo Mozzer Ferraz De Souza, and Antonio Carlos Arnoni Junior. "Machine learning for classification of it support tickets." In 2023 International Conference On Cyber Management And Engineering (CyMaEn), pp. 210-213. IEEE, 2023.
- [2]. Dlodlo, Dalubuhle, and Khulekani Sibanda. "Automated Ticket Classification for Information Technology Helpdesks using Machine Learning." In 2023 2nd Zimbabwe Conference of Information and Communication Technologies (ZCICT), pp. 1-7. IEEE, 2023.
- [3]. Towhid, Md Shamim, Nasik Sami Khan, Nashid Shahriar, Massimo Tornatore, Raouf Boutaba, and Aladdin Saleh. "A Token-Prioritization Strategy for Handling Data Imbalance in Network-Change Ticket Classification." In 2023 19th International Conference on Network and Service Management (CNSM), pp. 1-7. IEEE, 2023.
- [4]. C. Sun, S. Huang and D. Pompili, "LLM-Based Multi-Agent Decision-Making: Challenges and Future Directions," in IEEE Robotics and Automation Letters, vol. 10, no. 6, pp. 5681-5688, June 2025
- [5]. Tipe-Palomino, David, and Wilver Auccahuasi. "Development of Helpdesk Chatbot for Incident Classification and Resolution using NPL and Deep Learning." In 2024 3rd International Conference on Automation, Computing and Renewable Systems (ICACRS), pp. 785-791. IEEE, 2024.
- [6]. Anish, S., B. N. Rahul, KR Pavan Kumar, Rahul Sankar, and B. M. Mala. "Customer Support Ticket Generation Using RPA." In 2025 Control Instrumentation System Conference (CISCON), pp. 1-6. IEEE, 2025.
- [7]. Yang, Juan & Wang, Minjuan & Du, Xu & Na, Rina. (2025). A Comprehensive Survey on Large-Language-Model-Based Agents for Education. IEEE Transactions on Learning Technologies. PP. 1-17. 10.1109/TLT.2025.3617909.
- [8]. Adapa, Chathurya, A. R. K. Anjana, Rafsal Rahim, and Ajay Victor. "A Multi-Agent AI Framework for Agile Workflow Automation, Issue Resolution, and Developer Performance Evaluation." In 2025 IEEE International Conference for Women in Innovation, Technology & Entrepreneurship (ICWITE), pp. 1-6. IEEE, 2025.

- [9]. S. Yoon, "Strategic Learning Under Linguistic and Contextual Constraints: A Theoretical Framework for LLM-Based Multi-Agent Coordination," in IEEE Access, vol. 13, pp. 197053-197071, 2025
- [10]. Gamage, U. R., W. S. N. Fernando, and K. A. V. U. Kuruppu. "Generative AI Based Chatbot for Customer IT Support Automation." In 2025 7th International Conference on Advancements in Computing (ICAC), pp. 1-6. IEEE, 2025.
- [11]. Park, Daeseung, Gi-taek An, Chayapol Kamyod, and Cheong Ghil Kim. "A study on performance improvement of prompt engineering for generative AI with a large language model." Journal of web engineering 22, no. 8 (2023): 1187-1206.
- [12]. Pandey, Shivansh, Sandhya Avasthi, and Aditya Tandan. "Conversational Career Guidance System using LangChain and Agentic AI Framework." In 2025 IEEE International Conference on Communication Networks and Computing (CNC), pp. 481-485. IEEE, 2025.
- [13]. Gemayel, Johny, and Ali Mokh. "Network Function Orchestration with LLM based Multi-Agent System." In 2025 IEEE International Conference on Communications Workshops (ICC Workshops), pp. 262-267. IEEE, 2025.
- [14]. Poniszewska-Marańda, Aneta, Maciej Kopa, and Bożena Borowska. "Multi-agent systems for improved information retrieval-leveraging autonomous agents and LLM models." In 2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW), pp. 303-310. IEEE, 2025.
- [15]. Sudheer, Arjun, Chia-Hong Chou, and Shubham Kumar. "A Real-time Multi-Agent Network Attack Detection and Incident Response System." In 2025 Silicon Valley Cybersecurity Conference (SVCC), pp. 1-3. IEEE, 2025.
- [16]. Meruga, Vishal Bharadwaj. "LLM-Guided Multi-Agent Collaboration for Complex Task Automation." In 2025 5th International Conference on Emerging Research in Electronics, Computer Science and Technology (ICERECT), pp. 1-7. IEEE, 2025.